

The Family Metro Map Layout Algorithm

Design, beam search, scoring and overlap repair

Technical description of the implementation in `metro_core.py`
(identical results to `familymetromap.js`)

Abstract

This document describes how the application draws a tree as an octilinear “metro map” in the style of Korst, Pronk and van Wijk (2020). The layout is built generation by generation: for every vertex, the directions, bends and lengths of the links to all of its children are chosen together by a small *beam search* that scores every candidate by its conflicts, its clearance from the existing drawing, the free room it leaves for the child’s own subtree, and metro-style preferences. Remaining vertex and edge overlaps are then detected and, on request, repaired by a budgeted local search. We define every quantity used by the code, walk through a real beam-search run step by step, analyse the running time, and report experiments on beam width and look-ahead, including cases where the beam search does *not* help.

Contents

1	Overview	1
2	Orientation and link model	2
3	Length model	4
4	Beam search over the children of one vertex	4
5	The scoring function	8
6	Collision predicate and overlap detection	9
7	Overlap repair (the Fix buttons)	10
8	Experiments	11
9	Differences from the paper and limitations	12

1 Overview

The input is an undirected tree $T = (V, E)$ with a chosen root r . Every vertex v has a gender $\sigma(v) \in \{F, M\}$; in the application it is derived from the vertex label (the character code of its first letter is even for female). The output is a position $p_v \in \mathbb{R}^2$ for every vertex and, for some edges, one bend point. Figure 1 shows the complete pipeline.

The layout itself has four ingredients, each described in its own section:

1. the **orientation and link model** (section 2): which directions a link may take and when it needs a bend;
2. the **length model** (section 3): how long a link is, depending on its line generation;

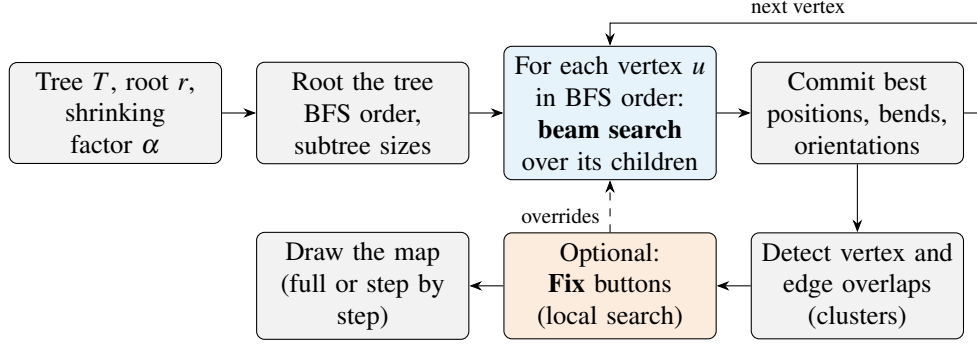


Figure 1: Pipeline. The layout (blue) is deterministic. The fixers (orange) do not move vertices directly; they record *overrides* (forced choices for individual links) and re-run the layout.

3. the **beam search** (section 4): how the choices for all children of one vertex are made together;
4. the **scoring function** (section 5): how one candidate placement is evaluated.

2 Orientation and link model

2.1 Octilinear directions and gender

All line segments are horizontal, vertical or diagonal. A direction is an integer $k \in \{0, \dots, 7\}$ meaning the angle $k \cdot 45^\circ$, with unit vector

$$\mathbf{e}_k = (\cos(k\pi/4), \sin(k\pi/4)).$$

Because screen coordinates grow *downwards*, $k = 2$ points down and angles turn clockwise on screen (fig. 2). Each vertex v receives an *orientation* $k(v)$: the direction of the last segment of the link that arrives at v . Following the paper, the orientation encodes gender:

$$k(v) \text{ is even} \iff \sigma(v) = \text{F} \quad (\text{horizontal/vertical}), \quad k(v) \text{ is odd} \iff \sigma(v) = \text{M} \quad (\text{diagonal}).$$

The root has no incoming link; it is given $k(r) = 2$ if female and $k(r) = 3$ if male.

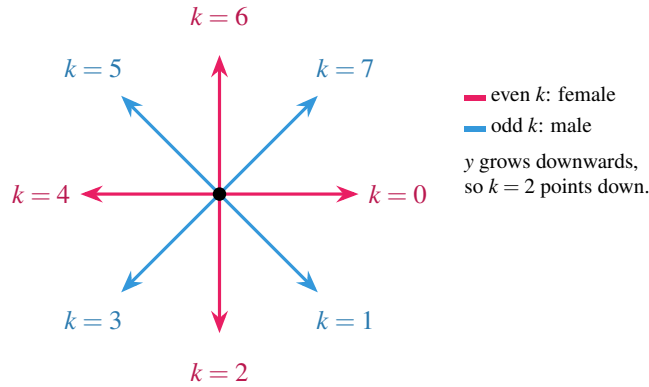


Figure 2: The eight octilinear directions. Even directions are used for female vertices and odd (diagonal) directions for male vertices.

2.2 Branch offset g and bend sign s

Let u be a vertex with orientation $k(u)$ and let v be one of its children. The link $u \rightarrow v$ leaves u with a *first-segment direction*

$$d = (k(u) + g) \bmod 8, \quad g \in \{0, +2, -2, +1, -1, +3, -3, 4\},$$

where the *branch offset* g has the following meaning:

g	meaning	in the paper?	preference rank $\pi(g)$
0	continue the parent's metro line straight on	yes	0
± 2	perpendicular T-junction (a new side line)	yes	1
± 1	diagonal branch at $\pm 45^\circ$	no (high degree only)	2
± 3	diagonal branch at $\pm 135^\circ$	no (high degree only)	3
4	opposite direction (only useful at the root)	no	4

The paper's family trees are binary (each person continues a line and starts at most one side line), so $g \in \{0, \pm 2\}$ suffices there. A general tree can have up to seven children per non-root vertex, so the diagonal offsets are added as extra branch directions; their higher preference rank makes the search use them only when needed.

Whether the link needs a bend depends only on whether d already has the right parity for v :

$$\text{straight link} \iff (d \text{ even}) = (\sigma(v) = F), \quad \text{otherwise a bended link with } s \in \{+1, -1\}.$$

The final orientation is $k(v) = d$ for a straight link and $k(v) = (d + s) \bmod 8$ for a bended link, so the gender rule always holds.

2.3 Link geometry

For a link of (Euclidean) length ℓ :

$$\text{straight: } p_v = p_u + \ell \mathbf{e}_d,$$

$$\text{bended: } b = p_u + \lambda \mathbf{e}_d, \quad p_v = b + \lambda \mathbf{e}_{d+s}, \quad \lambda = \frac{\ell}{\sqrt{2 + \sqrt{2}}} \approx 0.541 \ell.$$

The value of λ makes the distance $|p_v - p_u|$ exactly ℓ : the two segments of length λ meet at an interior angle of 135° , so by the law of cosines $|p_v - p_u|^2 = 2\lambda^2(1 - \cos 135^\circ) = \lambda^2(2 + \sqrt{2}) = \ell^2$. This is the construction of Section 4.1 of the paper. Figure 3 illustrates the three cases.

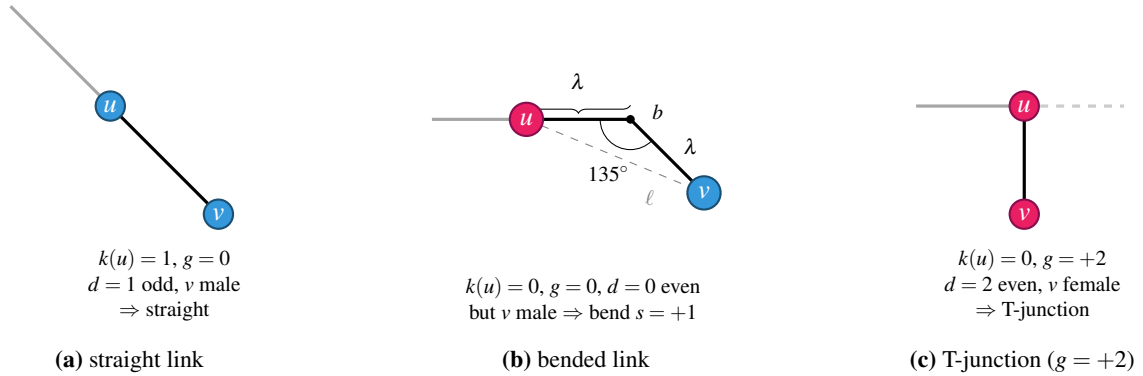


Figure 3: The three basic link types. Grey: the link arriving at u . The bended link has two segments of length $\lambda = \ell / \sqrt{2 + \sqrt{2}}$, so v is still at distance ℓ from u .

2.4 Distinct first directions

A key rule, which removes most edge overlaps at their source, is that *the links leaving a vertex use pairwise different first-segment directions*, and none of them uses the direction back along the incoming link, $(k(u) + 4) \bmod 8$. Two links that start in the same direction would share their first segment, which the overlap detector reports as an edge overlap. A non-root vertex therefore has 7 free directions and the root 8; a vertex with more children than free directions cannot be drawn without an edge overlap, and only then is the rule relaxed (section 4).

3 Length model

The paper draws all links of a metro line of generation g with the same length α^g , where $0 < \alpha \leq 1$ is the shrinking factor: the root line has generation 0, its side lines generation 1, and so on. Here, the *line generation* $\gamma(v)$ is defined along the tree:

$$\gamma(r) = 0, \quad \gamma(v) = \begin{cases} \gamma(u) & \text{if the link } u \rightarrow v \text{ continues the line } (g = 0), \\ \gamma(u) + 1 & \text{otherwise (a side line starts).} \end{cases}$$

The length of the link $u \rightarrow v$ is

$$\ell_{uv} = \max\left(\max(L_0 \alpha^{\gamma(v)}, L_{\min}) \cdot f_{uv} \cdot \mu_u, L_{\text{abs}}\right),$$

with base length $L_0 = 90$ px, floor $L_{\min} = 34$ px (so that the 16 px vertex disks are always well apart), and a hard floor $L_{\text{abs}} = 20$ px. The factors f_{uv} (length factor of one link) and μ_u (product of the local shrinking factors of all subtrees containing u) are 1 unless the fixers have changed them (section 7).

Note that the length depends on g through $\gamma(v)$: continuing a line keeps the longer length of the parent's line, while a side line is shorter by a factor α . This is one reason why continuing straight is often the cheapest option.

4 Beam search over the children of one vertex

4.1 The decision problem at one vertex

Vertices are processed in breadth-first order. When vertex u is processed, u itself and everything placed before it are fixed ("committed"). The task is to place all children $v_1, \dots, v_m$ of u , that is, to choose for each child an option

$$o_i = (g_i, s_i) \implies d_i, \ell_i, b_i, p_{v_i}, k(v_i),$$

such that the first directions d_i are pairwise distinct and different from the back direction, and such that the total cost

$$C(o_1, \dots, o_m) = \sum_{i=1}^m \text{score}(o_i \mid \text{committed geometry}, o_1, \dots, o_{i-1})$$

is small. The score of a child depends on the siblings placed before it (they can collide with it), so the choices interact. With up to 12 options per child (8 offsets, half of them with two bend signs), exhaustive search costs up to 12^m evaluations, which is too many for vertices with many children. A purely greedy choice (best option for v_1 , then best for v_2 given $v_1, \dots$) is cheap but may commit early to an option that leaves bad choices for later siblings. Beam search is the usual compromise between the two.

4.2 Candidate options

For each child v the candidate list is built in a fixed order (this order also breaks ties):

1. offsets g in the order $0, +2, -2, +1, -1, +3, -3, 4$ (or only the forced offset if a fixer set one);
2. for each offset: $s = 0$ if the link is straight; otherwise $s = +1$ then $s = -1$ (or only the forced sign).

With four even and four odd directions, exactly four offsets give straight links and four give bended links, so a child has $4 + 4 \cdot 2 = 12$ options. At a non-root vertex the offset $g = 4$ is the blocked back direction, which leaves 10 or 11 usable options (as in table 1, where J1 has 10). Figure 4 shows the options around a vertex.

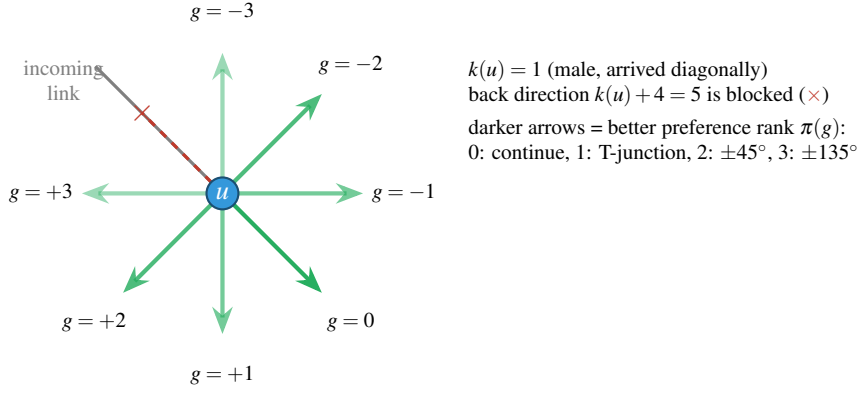


Figure 4: Candidate first directions at a non-root vertex. Each direction is used by at most one child. Whether a direction yields a straight or a bended link depends on the gender of the child.

4.3 The beam

A *state* is a partial assignment $(o_1, \dots, o_j)$ for the first j children together with its accumulated cost. The children are ordered by *decreasing subtree size* (ties by their original order), so the largest subtrees choose first and get the best directions.

With $B = 1$ this is exactly the greedy algorithm; with $B = \infty$ it is exhaustive search. Because the sort is stable and the option order is fixed, the result is deterministic: running the layout twice gives the same map, and the JavaScript and Python implementations produce identical coordinates.

4.4 A worked example

Table 1 shows a real run from the 100-vertex example tree ($\alpha = 0.75$, root A). Vertex I1 (male, orientation $k = 3$) has three children: J1 (female), Q1 and U2 (both male). Part of the neighbourhood is already crowded, so many options are penalised. The “penalty” column is the geometric part of the score (clearance and look-ahead, section 5); none of the listed options has a conflict.

Algorithm 1 PLACECHILDREN(u): beam search for the children of u (beam width $B = 4$)

```

1:  $v_1, \dots, v_m \leftarrow$  children of  $u$  sorted by subtree size (largest first)
2:  $blocked \leftarrow \{(k(u) + 4) \bmod 8\}$  if  $u \neq r$ , else  $\emptyset$ 
3: compute candidate options  $O_i$  for every  $v_i$  and the reach radius  $R$  (section 5.2)
4:  $\mathcal{B} \leftarrow \{(\text{empty assignment, cost } 0)\}$ 
5: for  $i = 1$  to  $m$  do
6:    $\mathcal{N} \leftarrow \text{EXPAND}(\mathcal{B}, O_i, \text{distinct} = \text{true})$ 
7:   if  $\mathcal{N} = \emptyset$  then  $\triangleright$  more children than free directions, or forced choices clash
8:      $\mathcal{N} \leftarrow \text{EXPAND}(\mathcal{B}, O_i, \text{distinct} = \text{false})$ 
9:   stable-sort  $\mathcal{N}$  by cost;  $\mathcal{B} \leftarrow$  the first  $B$  states of  $\mathcal{N}$ 
10: commit the best state of  $\mathcal{B}$ : positions, bends, orientations  $k(v_i)$ , generations  $\gamma(v_i)$ 

11: function EXPAND( $\mathcal{B}, O, \text{distinct}$ )
12:    $\mathcal{N} \leftarrow \emptyset$ 
13:   for all states  $S \in \mathcal{B}$  do
14:      $used \leftarrow blocked \cup \{\text{first directions used in } S\}$ 
15:     for all options  $o \in O$  do
16:       if  $\text{distinct}$  and  $d(o) \in used$  then continue
17:       compute  $p_v, b, k(v)$  for  $o$   $\triangleright$  section 2
18:        $c \leftarrow \text{score}(o \mid \text{committed}, S)$   $\triangleright$  section 5
19:       add  $(S \cup \{o\}, \text{cost}(S) + c)$  to  $\mathcal{N}$ 
20:   return  $\mathcal{N}$ 

```

rank	state (previous choices) $\rightarrow$ new option	d	penalty	pref.	avg. term	cost	
Level 1: child J1 (10 candidates, keep 4)							
0	$\emptyset \rightarrow \text{J1: } g = 0, s = +1$	3	0.0	0	-7.56	-7.56	kept (greedy pick)
1	$\emptyset \rightarrow \text{J1: } g = +1$	4	0.0	24	-7.23	16.77	kept
2	$\emptyset \rightarrow \text{J1: } g = +2, s = -1$	5	68.2	12	-7.03	73.14	kept
3	$\emptyset \rightarrow \text{J1: } g = -1$	2	523.8	24	-7.35	540.44	kept
4	$\emptyset \rightarrow \text{J1: } g = 0, s = -1$	3	719.2	0	-7.64	711.60	pruned
5	$\emptyset \rightarrow \text{J1: } g = -3$	0	1 000 803.8	36	-6.57	$\approx 10^6$	pruned (conflict)
Level 2: child Q1 (38 candidates, keep 4)							
0	[J1 $d=4$] $\rightarrow$ Q1: $g = -1, s = +1$	2	0.0	24	-7.44	33.33	kept
1	[J1 $d=3$] $\rightarrow$ Q1: $g = +1, s = +1$	4	68.2	24	-7.03	77.58	kept (greedy pick)
2	[J1 $d=5$] $\rightarrow$ Q1: $g = -1, s = +1$	2	0.0	24	-7.44	89.70	kept
3	[J1 $d=5$] $\rightarrow$ Q1: $g = 0$	3	101.1	0	-7.65	166.56	kept
4	[J1 $d=5$] $\rightarrow$ Q1: $g = +1, s = -1$	4	204.5	24	-7.37	294.24	pruned
Level 3: child U2 (32 candidates)							
0	[J1 5, Q1 2] $\rightarrow$ U2: $g = +1, s = -1$	4	204.5	24	-7.37	310.80	winner
1	[J1 4, Q1 2] $\rightarrow$ U2: $g = -3, s = -1$	0	380.8	36	-6.41	443.69	
2	[J1 3, Q1 4] $\rightarrow$ U2: $g = -1, s = +1$	2	380.8	24	-7.44	474.91	greedy result

Table 1: Beam-search trace at vertex I1. “pref.” is $12\pi(g)$ and “avg. term” is $-0.02 \times$ the average distance to the placed vertices. The cost of a state is the cost of its parent state plus the new option’s score.

The greedy algorithm would take the cheapest option for J1 (continue straight, -7.56), but this leaves Q1 and U2 only poor directions, ending at a total of 474.91. The beam keeps the third-best J1 option (73.14) alive; from it, Q1 and U2 find much better places and the total drops to 310.80. Figure 5 draws the same

search as a tree and fig. 6 shows the two resulting placements.

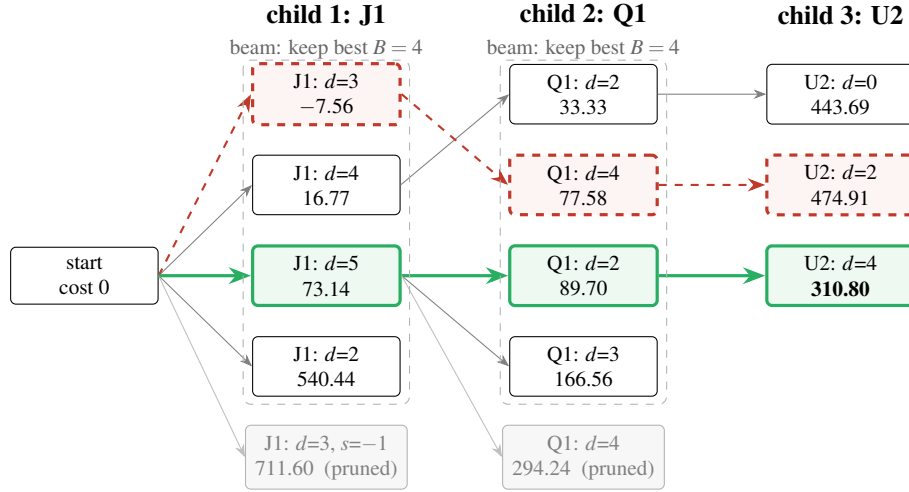


Figure 5: The beam search of table 1 (only a few of the expanded states are drawn). **Green:** the path to the winning assignment. **Red, dashed:** the path greedy search ($B = 1$) would follow. Grey boxes are pruned because they are not among the B cheapest states of their level.

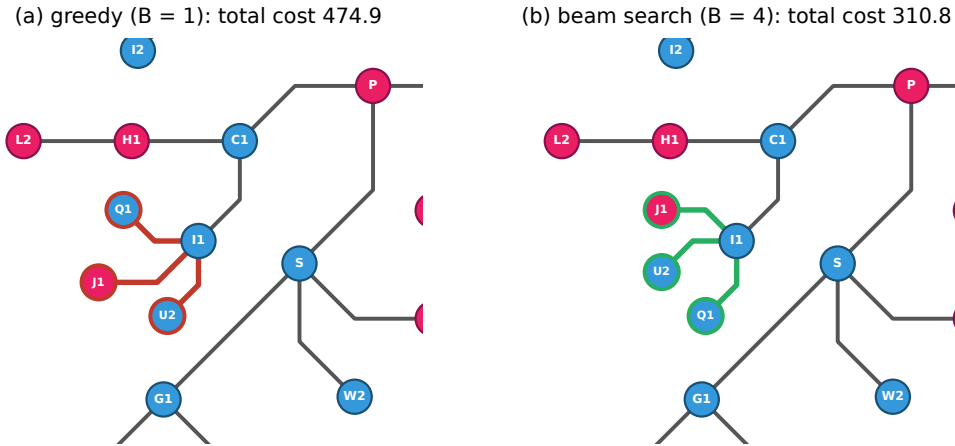


Figure 6: The two placements of I1's children from fig. 5, drawn on top of the geometry that was committed before I1 was processed. Greedy (a) squeezes J1, Q1 and U2 against the existing lines; the beam (b) finds an arrangement with more clearance.

4.5 Complexity

Let m_u be the number of children of u , $|O| \leq 12$ the number of options per child, and N_u, S_u the numbers of committed vertices and segments near u . Level i expands at most B states into at most $B|O|$ candidates, each scored in $O(N_u + S_u + i)$ time. Sorting adds $O(B|O| \log(B|O|))$ per level. The whole layout therefore costs

$$O\left(\sum_u m_u B|O|(N_u + S_u)\right) \subseteq O(B|O|n^2),$$

because $\sum_u m_u = n - 1$ and $N_u, S_u = O(n)$. The average-distance term (section 5) iterates over all placed vertices and is the only part that is always linear in n ; the rest is limited to the neighbourhood of u . In

practice the layout takes about 45 ms for a 45-vertex tree and 0.1–0.2 s for 100 vertices in Python (about 4–10 times less in JavaScript).

5 The scoring function

5.1 Definition

Let u be the parent, o the option for child v with resulting position p_v , optional bend b , final orientation $k(v)$ and length ℓ , and let Σ_{new} be the one or two new segments. “Existing” means committed geometry near u plus the tentative siblings of the current beam state. The score is

$$\text{score}(o) = 10^6 V + 10^5 E + 40 \max(0, 0.9\ell - c) + P_{\text{room}} + 12\pi(g) - 0.02\bar{\delta},$$

with the following terms (constants are summarised in table 2):

V – vertex conflicts

the number of (i) existing vertices $w \neq u$ with $|p_v - p_w| < 26$ px, (ii) pairs (existing vertex $w \neq u$, new segment) at distance < 12 px (a line running through a vertex), and (iii) existing segments at distance < 12 px from p_v (the new vertex sits on a line).

E – edge conflicts

the number of pairs (new segment, existing segment) for which the collision predicate of section 6 reports a crossing, a same-direction overlap, or a touch closer than 6 px.

c – clearance

$c = \min(\min_w |p_v - p_w|, \min_{\sigma} 1.5 \cdot \text{dist}(p_v, \sigma))$ over existing vertices $w \neq u$ and existing segments σ not incident to u ; if nothing exists, $c = \ell$. The penalty is only active below 0.9ℓ , so it pushes vertices apart without forcing an ever-larger spread. This is the analogue of the paper’s c_1 (minimum distance), extended to vertex–line distances, which the paper names as a weakness of c_1 alone.

P_{room} – look-ahead

only if v has children. Two sample points $q_t = p_v + t\ell \mathbf{e}_{k(v)}$ for $t \in \{0.6, 1.2\}$ lie where v ’s own line would continue. With ρ the smallest distance from a sample point to an existing vertex ($\neq u$) or a segment not incident to u ,

$$P_{\text{room}} = 15 w_v \max(0, \ell - \rho), \quad w_v = \frac{1}{2} \min(|T_v| - 1, 4),$$

where $|T_v|$ is the size of v ’s subtree. Bigger subtrees thus ask for more free space ahead.

$12\pi(g)$ – metro preference

continuing a line is free, a T-junction costs 12, diagonal branches 24 or 36.

$\bar{\delta}$ – spread

the average distance from p_v to all placed vertices, a mild pull outwards; the analogue of the paper’s c_2 .

The weights are chosen so that the terms act roughly lexicographically: any conflict ($\geq 10^5$) outweighs every geometric penalty; the geometric penalties are typically tens to hundreds and therefore usually dominate the preferences (at most 48); and the spread term (a few units) mainly breaks near-ties. Figure 7 illustrates the geometric terms.

5.2 Spatial pruning

Before the beam search at u starts, the committed vertices and segments are filtered to those within the reach radius $R = 2.4\ell_{\text{max}} + 36$ px of u , where ℓ_{max} is the longest candidate link of any child. All new segments lie within ℓ_{max} of u , so every conflict threshold (26, 12 and 6 px) and every clearance value below 0.9ℓ is unaffected by geometry outside R : those terms are computed exactly. The look-ahead samples reach up to 2.2ℓ from u , so for long links a far-away obstacle can be missed by P_{room} ; the pruning is therefore a slight approximation of the look-ahead term only. The spread term $\bar{\delta}$ always uses all placed vertices.

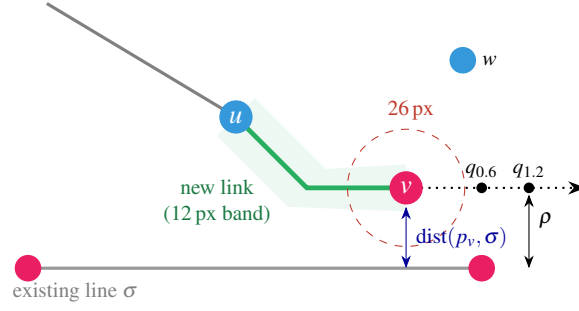


Figure 7: Geometric quantities used by the score of a new link $u \rightarrow v$: the 26 px vertex clearance circle, the 12 px band around the new segments, the distance to a foreign line σ (enters the clearance c with factor 1.5), and the look-ahead samples $q_{0.6}, q_{1.2}$ along v 's own direction with free room ρ .

constant	value	role
L_0	90 px	base link length (root line)
$L_{\min}, L_{\text{abs}}$	34 px, 20 px	length floors (normal, after fixer shrinking)
vertex clearance	26 px	vertex–vertex conflict threshold in the score
line clearance	12 px	vertex–line conflict threshold in the score
edge tolerance	6 px	touching edges (score and overlap detection)
vertex overlap	16 px	two disks overlap (detection only)
B	4	beam width
weights	$10^6, 10^5, 40, 15, 12, 0.02$	V, E , clearance, look-ahead, preference, spread

Table 2: Constants of the layout.

5.3 Relation to the paper's cost $C(M)$

The paper compares complete maps lexicographically by $C(M) = (c_1, c_2)$: first the minimum distance between non-neighbouring vertices, then the average distance, and it treats crossings as a hard constraint. The score here is local (one link at a time, against the geometry known so far) and additive (so states of a beam can be compared by a single number). Crossings are a very large penalty instead of a hard constraint, so a drawing is always produced, even for trees that admit no crossing-free map.

6 Collision predicate and overlap detection

Two segments $[p_1, p_2]$ and $[p_3, p_4]$ belonging to different edges are tested as follows (fig. 8):

1. **Bounding boxes.** If the boxes are more than 6 px apart, there is no collision (a pure speed-up).
2. **Shared endpoint** (within 1 px). The segments meet at a vertex or at a bend. They collide only if they leave the shared point in the same direction (angle difference < 0.05 rad); otherwise meeting is legitimate.
3. **Proper crossing.** If no endpoint is shared and the orientation tests show that each segment separates the endpoints of the other, they cross (an *overlap*).
4. **Touch.** Otherwise, if one segment comes within 6 px of the other (minimum of the four endpoint–segment distances), they *touch*.

After the layout, the overlap panels are computed on what is currently visible:

- **Vertices:** the top-left pixel positions are rounded as on screen, and two vertices closer than 16 px (the disk size) overlap.
- **Edges:** all pairs of drawn segments are tested with the predicate above.

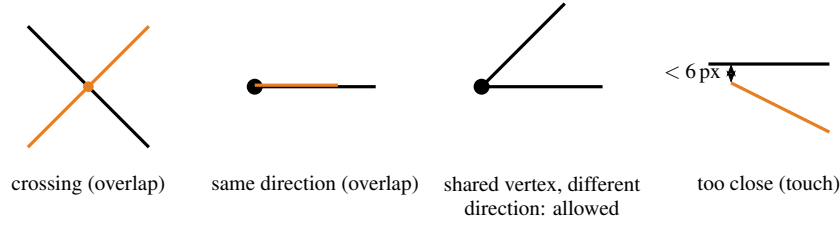


Figure 8: Cases of the edge collision predicate.

In both cases the pairs are merged into connected components, so that three mutually overlapping vertices are reported once as “Vertices B, H, N overlap”. During step-by-step drawing only vertices already drawn (and edges whose two endpoints are drawn) are checked, and components that did not exist in the previous step are marked NEW.

7 Overlap repair (the Fix buttons)

The layout is fast but local, so overlaps can remain, especially for small α or vertices of high degree. The two Fix buttons run a local search over *overrides*, i.e. choices that the layout must respect:

- a forced offset g and/or bend sign s for one link (the paper’s suggestion of “flipping” subtrees, done automatically);
- a length factor f_{uv} for one link;
- a local shrinking factor for the subtree below a vertex (a local version of Lemma 4.3 of the paper, which states that a small enough shrinking factor always separates neighbouring subtrees).

Because the layout is re-run after every change, everything placed after a modified link adapts automatically.

Algorithm 2 FIX(mode) with mode $\in \{\text{vertex}, \text{edge}\}$

```

1:  $best \leftarrow$  current overrides;  $e \leftarrow$  EVALUATE( $best$ )
2: for round = 1, ..., 16 while overlaps of the chosen kind remain and budget is left do
3:    $\mathcal{C} \leftarrow$  vertices involved in the overlaps, plus their parents and grandparents
4:   for all  $x \in \mathcal{C}$  with parent  $p$  do ▷ 1. re-route one link
5:     for all  $(g, s)$  in the trial list of link  $p \rightarrow x$  do
6:        $e' \leftarrow$  EVALUATE( $best + \{p \rightarrow x : g, s\}$ )
7:       remember it if  $\text{cost}(e')$  is the lowest so far (edge mode: and no new vertex overlaps)
8:       accept the best trial for  $x$  if it improves on the current cost
9:   if nothing improved then try length factors 0.76–1.3 for one link ▷ 2. rescale
10:  if nothing improved then try local shrinking 0.8, 0.65, 0.5 for one subtree ▷ 3. Lemma 4.3
11:  if nothing improved then stop
12: return  $best$  and a summary of every accepted change

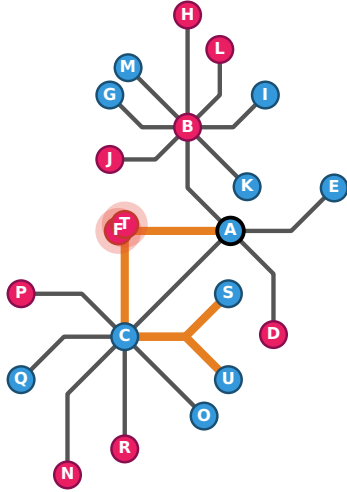
```

EVALUATE runs the full layout and computes

$$\text{cost} = 100000 \cdot (\#\text{vertex pairs closer than 16 px}) + 5000 \cdot (\#\text{colliding edge pairs}) - (\text{minimum vertex distance}).$$

The trial list for a link contains every offset $g \in \{\text{free}, +2, -2, 0, +1, -1, +3, -3\}$ (plus 4 at the root), each with $s \in \{\text{free}, +1, -1\}$ when the link is bended; variants that would not change anything are skipped. The search is deterministic and limited to $\max(150, \min(500, \lfloor 25000/n \rfloor))$ evaluations. Figure 9 shows its effect on a tree in which vertex C has eight children and therefore only seven free directions: one edge collision is unavoidable and is reported as unresolved.

(a) after the layout algorithm



(b) after Fix Vertex + Fix Edge

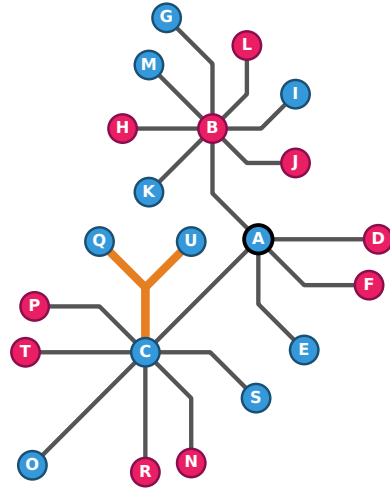


Figure 9: A 21-vertex test tree ($\alpha = 0.75$). (a) After the layout: vertices F and T overlap, and two groups of edges collide (orange). (b) After Fix Vertex Overlaps (“Relocated vertex F: branched as T-junction”) and Fix Edge Overlaps (two re-routed links): only C–Q and C–U still share their first segment, because C has more children than free directions.

8 Experiments

8.1 Effect of the design choices on overlaps

The orientation model, distinct first directions and line generations were introduced to replace an earlier version that placed all children of a vertex relative to the same base direction and shrank every BFS level by α . On 100 random trees with 10–45 vertices (at most 4 children per vertex), the total numbers of overlapping vertex pairs / colliding edge pairs after the layout alone were:

	$\alpha = 1.0$	$\alpha = 0.75$	$\alpha = 0.5$
earlier version	540 / 725	508 / 737	3027 / 2661
current layout	0 / 7	2 / 3	12 / 0

8.2 Beam width and look-ahead

To see what the beam search and the look-ahead contribute, the layout was run with beam widths $B \in \{1, 2, 4, 8\}$ and with the look-ahead term switched off, on three sets of random trees. Table 3 reports overlapping vertex pairs / colliding edge pairs (summed over the set) and, in brackets, the number of trees drawn without any overlap.

8.3 Discussion

The results are mixed, and worth stating plainly:

- The **look-ahead term helps** generally: removing it from $B = 4$ increases the number of collisions in almost every row, often substantially (e.g. 47 instead of 9 edge collisions on set L at $\alpha = 0.75$).
- The **beam search optimises its local objective better** than greedy search (as in the example of table 1), and at $\alpha = 1$ it yields more overlap-free trees on the larger sets.
- However, for $\alpha \leq 0.75$ on large or dense trees, **greedy search ($B = 1$) produced fewer overlaps** than the default $B = 4$. Lower local cost does not always translate into a better final map: the summed

set	α	$B = 1$ (greedy)	$B = 2$	$B = 4$ (default)	$B = 8$	$B = 4$, no look-ahead
S: 100 trees, 10–45 vertices	1.0	0/2 (98)	0/3 (99)	0/5 (97)	0/5 (96)	0/5 (97)
	0.75	0/0 (100)	0/0 (100)	1/2 (98)	2/2 (97)	4/2 (96)
L: 24 trees, 60–100 vertices	1.0	0/31 (7)	0/40 (10)	0/27 (12)	0/45 (9)	0/32 (11)
	0.75	4/2 (19)	3/4 (20)	21/9 (14)	19/24 (13)	30/47 (11)
	0.5	17/4 (15)	19/6 (14)	80/34 (4)	76/28 (4)	71/59 (3)
H: 60 trees, 30–60 vertices, up to 6–8 children	1.0	0/66 (35)	0/44 (40)	0/49 (38)	0/49 (39)	0/50 (34)
	0.75	3/15 (49)	3/19 (50)	23/21 (45)	25/26 (43)	26/40 (34)
	0.5	5/6 (52)	11/9 (47)	54/17 (45)	73/42 (43)	79/52 (31)

Table 3: Overlapping vertex pairs / colliding edge pairs after the layout (no fixers), and the number of overlap-free trees in brackets. Running time grows roughly linearly with B : per tree about 9, 13, 17, 28 ms on set S and 95, 125, 190, 320 ms on set L.

score lets the search trade the free room of the largest subtree (placed first) for small gains such as preference points or spread of later siblings, and those trade-offs only show their consequences when the grandchildren are placed. Larger beams ($B = 8$) do not fix this.

Possible improvements suggested by these findings are: comparing beam states lexicographically (conflicts, then clearance of the largest subtree first) instead of by a single sum; weighting the look-ahead more strongly for large subtrees; or simply running the layout with both $B = 1$ and $B = 4$ and keeping the map with the better global cost, which at most doubles the (small) layout time. The remaining overlaps are in any case handled by the Fix buttons.

9 Differences from the paper and limitations

- The input is a general tree, not the paper’s binary family tree with couples and age-ordered children; “continuing a metro line” is implicit ($g = 0$).
- The paper’s *Clockwise* heuristic searches pairs of neighbouring lines in clockwise order with exhaustive sub-searches; here a per-vertex beam search in BFS order is used.
- Crossings are penalised rather than forbidden, and the cost is a weighted local score rather than the global lexicographic $C(M)$.
- The fixers may change single link lengths or shrink individual subtrees, which breaks the paper’s equal spacing within a generation.
- The exhaustive (exact) search that the paper proposes for small trees is not implemented.

Reference. J. Korst, V. Pronk and J. J. van Wijk. A Visualization of Family Relations Inspired by the London Metro Map. In *The 13th International Symposium on Visual Information Communication and Interaction (VINCI 2020)*, Eindhoven, The Netherlands. ACM, 2020. <https://doi.org/10.1145/3430036.3430065>

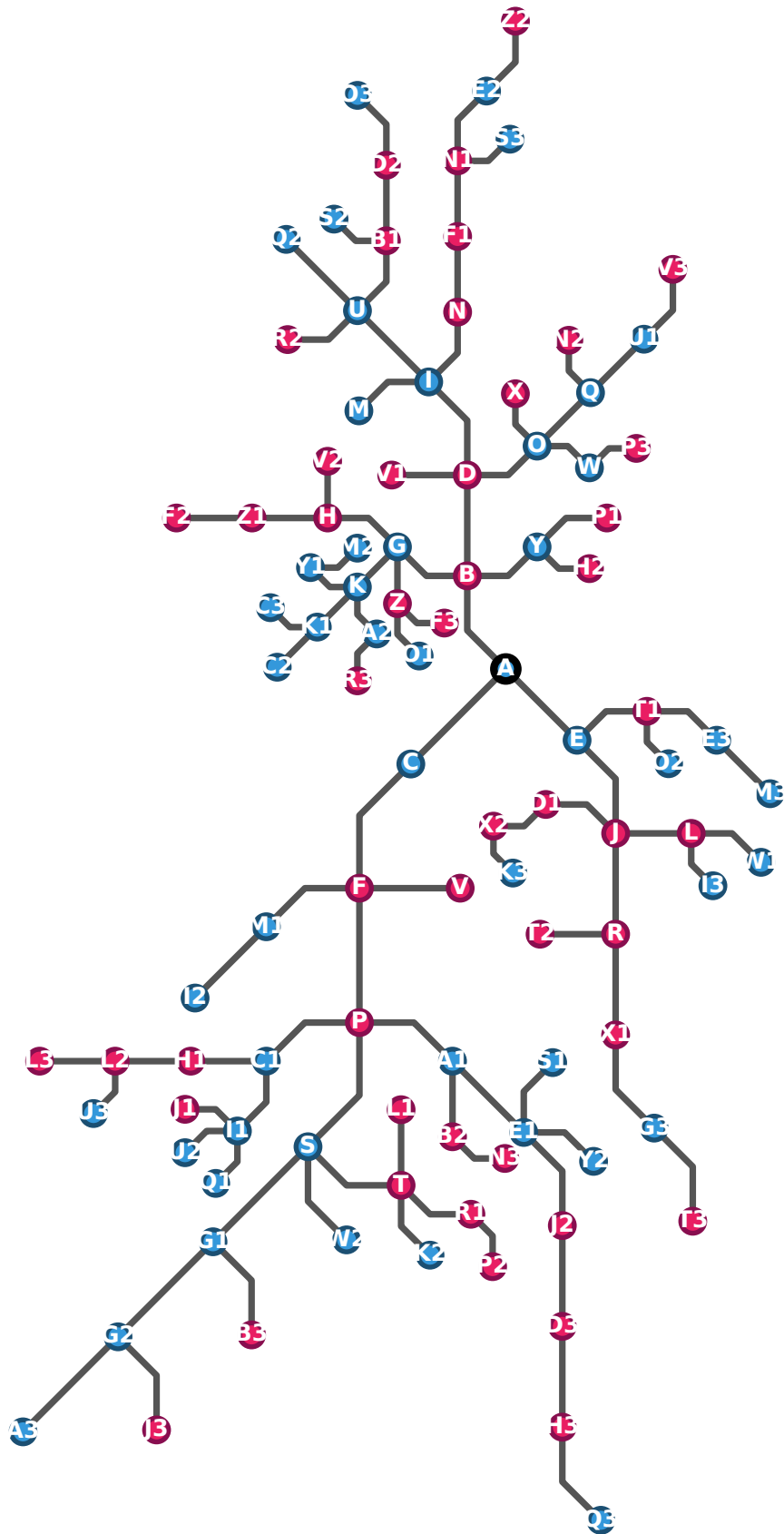


Figure 10: The 100-vertex example tree drawn by the layout algorithm ($\alpha = 0.75$, root A with black border), without any fixer; it has no vertex or edge overlaps.